

PLP Admissions — Full System Documentation

What the system does · who can do what · how every state change is processed

Verified against `baseline` after the latest project zip merge

Project: Pamantasan ng Lungsod ng Pasig (PLP) Admissions System

Repository: `github.com/kyoshiroo0123/plp_admissions` **Verified against:**

`baseline` branch **Audience:** project owners, presenters, evaluators, new developers, system administrators

This document explains what the system does, who can do what, and how every state change is processed. Every claim is derived from the actual code path. Where the system makes a deliberate trade-off, the rationale is included.

1. System Overview

1.1 Purpose

The PLP Admissions System is a web application that runs the entire student admissions cycle for Pamantasan ng Lungsod ng Pasig — from the moment a prospective student first registers, all the way through entrance-exam, interview, admission decision, and enrollment confirmation.

It replaces the manual, paper-based admissions workflow with a digital pipeline where every step is tracked, audited, and (where possible) automated.

1.2 What the system solves

Pain point in the old manual process	How the system solves it
Lost / misplaced paper documents	Documents are uploaded and stored centrally with audit-trail per status change.
Long lines and double-booking	Exam rooms and interview slots are auto-assigned by department + fairness rules.
Inconsistent results communication	In-app + email notifications on every status change.
No audit trail for decisions	Every state change is logged in <code>audit_logs</code> with actor, IP, timestamp.
Manual capacity tracking	Course caps and room capacities are enforced at the database layer.
No way for students to check status	A self-service student dashboard shows their progress in real time.

1.3 Workflow summary

Register → Verify email → Upload documents → Take entrance exam → Attend interview → Get result → Confirm enrollment.

Each transition is triggered by an event (form submission, staff approval, exam pass / fail) and produces side effects (status update, audit row, notification, downstream automation).

1.4 Main modules / features

Module	Purpose
Authentication	Login, registration, email verification, password reset, session management.
Documents	Student document upload, staff review and approval, request-resubmission flow.
Exam	Exam builder, slot scheduling, slot cancel & move, student-initiated reschedule requests, exam-taking interface, auto-grading.
Interview	Session setup, automatic slot assignment, live queue, evaluation recording, slot cancel & move.
Results	Decision release (single, bulk, auto, Close Admissions), override-reason gate, enrollment intent tracking.
Settings	Admin (school year, courses, caps, users), staff (profile), student (profile).
Dashboard	Per-role landing pages with funnel statistics and quick actions.
Audit	Append-only log of every state-changing action.
API	AJAX endpoints (notifications, exam autosave, reschedule requests, applicant panel).

2. User Roles and Access Levels

The system supports **six distinct roles**. Each role's permissions are enforced at three layers:

1. **Middleware** in `core/Auth.php` redirects unauthorized users.
2. **Per-route guards** (`Auth::requireRole(...)`) reject the request server-side.
3. **The UI** hides controls the user cannot use.

2.1 Admin

Description. The system owner. Has full control over configuration, user provisioning, and all admissions data across all colleges.

Can:

- **Create:** users (any role), courses, exam definitions, interview sessions in any college, course caps, school-year settings.
- **Read:** every applicant, every document, every exam result, every interview row, every result decision, every audit log.

- **Update:** any user (including roles, password reset, deactivation), school-year window, document deadline, course master data, system settings, branding, every automation toggle.
- **Delete:** users (soft / hard), courses (with cascade warning), exam rooms, interview sessions, system records.
- **Release results:** including **Auto Release** and **Close Admissions** (Admin-only).
- **Override:** any other role's decision (override-reason modal still fires when contradicting a recommendation).
- **Export:** CSV reports for applicants, results, exam statistics.

Cannot:

- Edit historical audit log rows (append-only by application convention).
- Edit a student's exam answers after submission (recorded immutably in `exam_results`).

Notes.

- Admin lands on `/admin/dashboard` after login.
- Admin is the only role that can change applicants' school year assignment after creation.
- Admin can use `?college=__all__` on the live interview queue to see every department simultaneously.

2.2 SSO (Student Services Office)

Description. The central admissions office. Owns the document review queue and coordinates exam / interview operations. Has system-wide read access but **does not release results** (role redesign).

Can:

- **Create:** exam rooms, exam definitions, course suggestions, manual exam-slot or interview-slot reassignments.
- **Read:** every applicant across all colleges, all documents, all exam results, all interview rows, all admission results.
- **Update:** document statuses (approve / request resubmission), applicant exam slot assignment, applicant interview slot assignment.
- **Delete:** exam rooms (own or admin-created), interview sessions in any college (with safety checks).
- **Approve / Request Resubmission:** documents (single and bulk).
- **Cancel & Move:** exam slots (`/staff/exam/cancel-slot`) and interview slots (`/staff/interviews/cancel-slot`).
- **Exam Reschedule queue:** approve / deny student-initiated reschedule requests at `/staff/exam/reschedule`.
- **Export:** all CSV reports.

Cannot:

- Release admission results (locked out of `/staff/results` in the role redesign).
- Create or delete users.
- Edit the school-year window or master course list.
- Edit system settings.

Notes.

- SSO is the only role that should normally use the document-review queue.
- SSO lands on `/admin/dashboard` after login (shares layout with Admin but with restricted side-nav).

2.3 Dean

Description. The head of a college (e.g., Dean of CCS). Per-college oversight. Can see and act on only their own college's applicants for interview and results, but participates in the global document-review pool.

Can:

- **Create:** interview sessions within their college, course-cap adjustments for courses in their college.
- **Read:** every applicant whose `course_applied` maps to their college, every interview slot in their college, all admission results in their college, audit logs (read-only).
- **Update:** documents (in the global doc-review queue), interview slots in their college, admission results for their college's applicants, course caps and tier thresholds (`avg_from` , `high_from`) for courses in their college.
- **Delete:** interview sessions in their college (with safety checks).
- **Approve / Request Resubmission:** documents (any college, since doc review is global).
- **Release results** for their college's applicants (single + bulk).
- **Cancel & Move:** interview slots in their college.
- **Export:** filtered CSV reports for their college.

Cannot:

- See or act on another college's interview queue.
- Release results for another college's applicants.
- Edit master course definitions (only `max_slots` + thresholds).
- Create or delete user accounts.
- Edit system settings or school-year window.
- Run **Auto Release** or **Close Admissions** (Admin-only).

Notes.

- Dean lands on `/admin/dashboard` after login.

- Dean's `department` is set on their `users` row and is the source of truth for per-college scoping on the interview queue and results page.
- The Dean is the **release authority** for their college in the role redesign. SSO is locked out of Results.

2.4 Staff (Professor / Interviewer)

Description. A faculty member responsible for interviewing applicants in their college. Most narrowly scoped of the staff roles.

Can:

- **Create:** interview sessions in their own college (always assigned to themselves by default).
- **Read:** their own assigned interview sessions, the queue rows for their sessions, the applicant card for any applicant in their queue.
- **Update:** their own interview sessions (date, time, capacity, location), the queue rows on interview day (Evaluate Pass / Decline). The queue is first-come, first-served by `queue_number` — there is no “Call Next” button and no manual “No-show” button in the UI.
- **Delete:** their own future interview sessions (cannot delete sessions with checked-in applicants).
- **Approve / Decline:** interview evaluations — stored as `pass` / `reject`. **Recommendation only** — the Dean releases the result.
- **Export:** filtered CSV report of their own queue.

Cannot:

- See other professors' queues (even within the same college).
- Create or modify interview sessions in other colleges.
- Approve or reject documents.
- Release admission results.
- Access admin / SSO / dean dashboards.

Notes.

- Staff lands on `/staff/dashboard` after login (different URL than Admin / SSO / Dean).
- Staff cannot manually pull applicants into their queue — `assign_interview_slot()` does that automatically when a student passes the exam, and `bulk_assign_pending_applicants()` sweeps waiting applicants into newly-created sessions.
- “Staff” and “Professor” are the same role internally (`ROLE_PROFESSOR` is an alias of `ROLE_STAFF`).
- The interview-evaluation outcome is **Pass / Decline**, not Pass / Fail.

2.5 Proctor (NEW)

Description. A thin operational role for exam day. Their sidebar (`views/partials/nav_proctor.php`) shows only **Dashboard** and **Exam Slots** — nothing else.

Can:

- **Read:** their own college's exam rooms.
- **Generate / Extend / New access code** for rooms in their own college.
`generate_exam_password()` returns a 6-character code valid for 5 minutes (`EXAM_PASSWORD_EXPIRY_SECONDS = 300`). All three actions are audited (`exam_slot_code_generated` , `exam_slot_code_extended`).

Cannot:

- Touch documents, exam build, interview queue, results, or admin pages.
- Generate access codes for rooms outside their college (gate enforced in `staff_slots.php`).

Notes.

- The Proctor lands on `/staff/dashboard` after login (same URL as Staff). The sidebar is filtered to two items.
- The role exists because exam-day operations were previously bundled into “Staff” and that scope was too wide — Proctor is now a dedicated, minimum-privilege role for that single responsibility.

2.6 Student (Applicant)

Description. The end user — a prospective applicant. Self-service portal limited to their own data.

Can:

- **Create:** their own account (via registration), upload their own documents, submit their own exam answers, submit a reschedule request for their exam (`/api/exam-reschedule-request`) or interview (`/api/reschedule-request`), confirm or decline enrollment.
- **Read:** their own applicant profile, their own documents, their own exam slot + exam questions (only when active), their own interview slot + queue position, their own admission result.
- **Update:** their own password and basic profile info (in `/student/settings`), their own documents before SSO has approved them (re-upload, change applicant type before submission).
- **Delete:** withdraw their application (sets status to `withdrawn` , terminal).

Cannot:

- See or interact with any other applicant's data.
- See staff-side dashboards, queues, or reports.

- Edit their exam answers after submission.
- Approve, reject, or modify their own admission status.

Notes.

- Student lands on `/student/documents` after login (or `/verify-pending` if email isn't verified yet).

3. Complete System Flow

The single source of truth for the end-to-end flow is the companion **Full Flow & Reference** document. The summary below is the abbreviated version of the same content.

3.1 Registration

`POST /register` (`modules/auth/register.php`):

1. Inputs are sanitized and normalized (names uppercased, email lowercased, etc.).
2. hCaptcha is verified against the hCaptcha API.
3. The admissions window is checked (`admissions_is_open()`); closed → reject.
4. Field validation (name, birthdate, sex, address, phone, email, password ≥ 8 chars, applicant type, course, SHS strand for freshmen).
5. **Course-cap pre-check.** `accepted_count ≥ max_slots` → block.
6. **Strand compatibility check** for freshmen.
7. **Duplicate detection** (same `first_name + last_name + birthdate`).
8. `BEGIN TRANSACTION` →
 - `INSERT INTO users` with `role = 'student'`, `department = course_to_department(course)`, password bcrypt-hashed (cost 12).
 - `INSERT INTO applicants` with `overall_status = 'pending'`.
 - Pre-create `documents` rows in `status = 'pending'`.
9. `COMMIT`.
10. Outside the transaction: generate verification credential pair (magic-link token + 6-digit code), send verification email, NOT auto-logged in → redirect to `/verify-pending`.

3.2 Email Verification

`/verify-pending` (`modules/auth/verify_pending.php`):

1. On success: `email_verified = 1`, all verification columns cleared, `Auth::login($user)` populates the session, login attempts cleared, redirect to `/student/documents`.

2. The magic link in the email (`/verify-email?token=...`) does the same.
3. Failure increments `email_verify_attempts`.

3.3 Login

`modules/auth/login.php`:

1. POST receives email + password + (optional) hCaptcha.
2. `is_locked_out($email)` checks `login_attempts` for the last 15 minutes.
3. If locked: "Too many failed attempts. Please wait 15 minutes."
4. Else: fetch user by lowercased email, verify password via `password_verify()`.
5. Wrong password → insert into `login_attempts`, render generic error.
6. Correct:
 - If `role = student` and `email_verified = 0`: redirect to `/verify-pending`.
 - Else: regenerate session ID, set last-active timestamp, redirect to `Auth::homeUrl()`.

3.4 Document Submission

`modules/documents/student_upload.php`:

1. Student sees a list of required documents (varies by applicant type — see Phase 3 in **Full Flow & Reference**).
2. For each row: **Choose File** → **Upload**.
3. Optionally **Change Applicant Type** if they registered as the wrong type (only allowed before submission).
4. When all required documents are at `uploaded` (or `approved`), click **Submit Application** → `overall_status = 'submitted'`.

Per-upload sequence:

1. The uploaded file is stored under `public/uploads/documents/{applicant_id}/`.
2. `UPDATE documents` sets `file_path`, `status = 'uploaded'`, `uploaded_at = NOW()`.
3. A basic file-type / size / integrity check runs. The OCR-style auto-validation pipeline and the `api/auto-validate` endpoint have been **retired in this build**; uploads still go through the basic integrity check.
4. Audit log row inserted.

3.5 Document Review

`modules/documents/staff_review.php` + `staff_action.php`:

1. `UPDATE documents` sets `status`, `reviewed_by`, `reviewed_at`, `staff_remarks`.
2. Count remaining unapproved required documents.

3. If count = 0:

- `UPDATE applicants SET overall_status = 'exam', documents_approved_at = COALESCE(...)`.
- Call `notify_stage_transition($id, 'exam')`.
- Call `auto_assign_exam_slot($id)`.

4. Audit log row inserted.

Per-document actions are **Approve** and **Request Resubmission**. There is no plain “Reject” button.

3.6 Exam Slot Assignment

1. Triggered when all documents are approved (Step 3.5).
2. `auto_assign_exam_slot($applicantId)` looks for the next exam room in the applicant’s department, ordered by date + remaining capacity.
3. Inserts into `applicant_exam_slots` (UNIQUE on `applicant_id` — prevents double-assignment).
4. Sends notification.

After a new slot is created from `/staff/exam/slots`, `backfill_exam_slot_assignments()` silently sweeps any waiting applicants in the department into the new slot.

3.7 Exam Reschedule (NEW)

- Student requests at `/student/exam` → POST `/api/exam-reschedule-request`.
- Inserts a `pending` row into `exam_reschedule_requests`.
- SSO / Admin reviews at `/staff/exam/reschedule` — Approve (assigns to replacement slot) or Deny (with reason).

3.8 Exam Slot Cancel & Move (NEW)

- SSO / Admin opens `/staff/exam/cancel-slot`.
- Picks source slot + replacement slot + reason.
- Submission locks both slots `FOR UPDATE`, re-validates capacity, moves every applicant in one transaction, fires notifications.

3.9 Entrance Exam (Taking)

`modules/exam/take.php`:

1. Renders one of three states depending on slot timing (no slot, future slot, today’s slot).

2. On exam day: access code gate (6 characters, valid for 5 minutes) → exam begins.
3. Anti-cheating: text selection disabled; timer counts from start to end; autosave every 60 s to `/api/exam-autosave` → `exam_drafts`.

On submit:

1. Each auto-gradable question is scored against the correct answer.
2. Free-text questions (paragraph) are not auto-graded.
3. `raw_score`, `percentage`, `rank = ceil(percentage / 10)` (clamped 1..10),
`passed = rank >= course_passing_scores.pass_from` (which is `avg_from` in code).
4. `INSERT INTO exam_results`.
5. If `passed = 1`:
 - `UPDATE applicants SET overall_status = 'interview'`.
 - Call `assign_interview_slot()`.
6. If `passed = 0`:
 - Status stays at `exam`.
 - `suggest_alt_courses()` finds courses with lower `pass_from`.

3.10 Interview Slot Assignment

`core/interview_scheduler.php :: assign_interview_slot()`:

1. Resolves department from `users.department` first, falling back to `course_to_department(course_applied)`.
2. `BEGIN TRANSACTION`.
3. `SELECT ... FOR UPDATE` locks all candidate slots in the applicant's department, ordered by `booked ASC, slot_date ASC, slot_time ASC, id ASC`.
4. Picks the first slot that has remaining capacity.
5. Refuses if the applicant already has an active queue row (UNIQUE on `interview_queue.applicant_id`).
6. `INSERT INTO interview_queue` with `status = 'checked_in'`, `queue_number = next sequential per slot`, `checked_in_at = NOW()`.
7. `UPDATE interview_slots SET booked = booked + 1`.
8. `COMMIT`.

3.11 Interview Day

`modules/interview/staff_queue.php`:

- Scoped per role (see Section 5 of **Full Flow & Reference**).
- Queue ordering is automatic — `in_progress` rows on top, then `checked_in` / `scheduled` by `queue_number ASC`, then `completed` / `no_show` at the bottom. **There**

is no **“Call Next” button** in the UI; the next applicant is whoever sits at the top. The legacy `staff_call_next.php` endpoint is dead code.

- **Auto no-show:** every page load runs an UPDATE that flips any still-waiting / in-progress row past its slot’s end time to `status=no_show` + `interview_status=absent` + `attendance_status=absent`. **No manual “No-show” button** in the UI.
- **Auto-reschedule:** auto no-shows are routed to the next available slot via `auto_reschedule_noshow()` when `auto_reschedule_noshows = '1'` (default on).
- **Evaluate** records `evaluation_result` (Pass / Decline — stored as `pass` / `reject`), sets `interview_completed_at`, AND flips `overall_status = 'released'` so the applicant shows up on Results.

3.12 Interview Slot Cancel & Move (NEW)

Mirror of the exam-side flow. SSO + Dean + Admin can open `/staff/interviews/cancel-slot`, pick a source + replacement, supply a reason, and submit. Same `FOR UPDATE` locking and notifications.

3.13 Results Release

`modules/results/staff_manage.php` (Auth: `ROLE_DEAN`, `ROLE_ADMIN`):

Three ways to release:

1. **Single row** — Accept or Reject button on `/staff/results`. If the decision contradicts the Professor’s recommendation, the override-reason modal demands a written reason.
2. **Bulk** — checkbox selection + Accept Selected / Reject Selected.
3. **Auto Release** (Admin-only) — sweep that decides based on exam + interview outcomes (only runs when `school_settings.auto_release_results = '1'`).
4. **Close Admissions** (Admin-only) — irreversible bulk-reject of every remaining unreleased applicant, with a recorded reason per row.

Single release transaction:

1. `BEGIN`.
 2. `INSERT INTO admission_results (applicant_id, result, remarks, released_by, released_at)`.
 3. `UPDATE applicants SET overall_status = 'released'`.
 4. `COMMIT`.
 5. Send in-app + email notification.
 6. Audit log row (`admission_result_released` or `admission_result_override`).
-

4. Module Reference

4.1 Authentication Module

- **Files:** `modules/auth/login.php`, `register.php`, `verify_pending.php`, `verify_email.php`, `forgot_password.php`, `reset_password.php`, `keepalive.php`, `logout.php`.
- **Helpers:** `core/Auth.php`, `core/Session.php`.

4.2 Documents Module

- **Files:** `modules/documents/student_upload.php`, `staff_review.php`, `staff_action.php`.
- **Helpers:** `docs_for_type()` in `config/app.php`.

4.3 Exam Module

- **Files:** `modules/exam/take.php` (student), `staff_manage.php` (build), `staff_slots.php` (rooms), `staff_reschedule.php` (reschedule queue), `staff_cancel_slot.php` (cancel & move), `staff_export_rooms.php`.
- **Student-side:** `modules/exam/student_reschedule.php` posts the request form data.
- **Helpers:** `generate_exam_password()`, `score_to_rank()`, `exam_passed()` in `core/helpers.php`.

4.4 Interview Module

- **Files:** `modules/interview/staff_setup.php`, `staff_queue.php`, `staff_action.php`, `staff_absent.php`, `staff_cancel_slot.php`, `student_view.php`. (`staff_call_next.php` exists but is dead code — no UI references it.)
- **Helpers:** `core/interview_scheduler.php`.

4.5 Results Module

- **Files:** `modules/results/staff_manage.php`, `staff_action.php`, `staff_bulk.php`, `staff_auto_release.php`, `staff_close_admissions.php`, `staff_suggest.php`, `enrollment_intent.php`, `student_view.php`, `admin_export.php`.
- **Auth:** `Auth::requireRole(ROLE_DEAN, ROLE_ADMIN)` on the staff manage page; Auto Release + Close Admissions are gated to `ROLE_ADMIN`.

4.6 Notifications Module

- **Files:** `modules/api/notifications.php`, `notify_stage_transition()` in `core/automation.php`, `core/mailer.php` (PHPMailer wrapper).
- **Tables:** `notifications` (in-app), email is delivered via PHPMailer + Gmail SMTP.

4.7 Audit Module

- **Files:** `modules/audit/log.php` (read-only viewer, Admin-only).
- **Helpers:** `audit_log()` in `core/helpers.php`.
- **Table:** `audit_logs` (`actor_id`, `action`, `description`, `entity`, `entity_id`, `ip_address`, `created_at`).

4.8 User Management Module

- **Files:** `modules/settings/admin_users.php`.
- **Auth:** Admin only.

4.9 Reports Module

- **Files:** `modules/auth/admin/dashboard.php`, `modules/results/admin_export.php`.
- **Output:** Chart.js bar charts + CSV exports.

4.10 Course Suggestion Module (NEW)

- **Files:** `modules/results/staff_suggest.php`, `modules/api/applicant_panel.php`.
- **Table:** `course_suggestions`.
- **Flow:** SSO / Dean / Admin picks an alternative course for a failed-exam applicant. Validates the applicant's rank against the suggested course's `pass_from`. The student sees the suggestion on `/student/result` and can Accept (switches `course_applied` and rolls them back into the pipeline) or Decline.

5. Database Flow

See the companion **Database Q&A** document for the exhaustive schema reference. Summary below.

5.1 Main tables (~26 schema-declared + ~5 on-demand)

```
users
departments
course_departments
department_schedules
applicants
documents
exams
exam_sections
questions
exam_results
exam_slot_schedule
applicant_exam_slots
interview_slots
interview_queue
reschedule_logs
admission_results
course_suggestions
course_caps
course_passing_scores
custom_courses
school_settings
audit_logs
password_resets
sessions
notifications
document_validations
```

On-demand auxiliaries: `exam_drafts`, `login_attempts`, `reschedule_requests`, `exam_reschedule_requests`, and re-declarations of `notifications` (idempotent).

5.2 Status state machine

```
pending → documents → submitted → exam → interview → released
                                                    ↘
                                                    withdrawn (terminal)
```

Every transition is gated by a `WHERE overall_status ...` clause that rejects out-of-order updates.

5.3 How data moves

- **Registration** → INSERT into `users` + `applicants` + `documents` in one transaction.
- **Document approval** → UPDATE `documents`, optionally UPDATE `applicants` + INSERT into `notifications` + INSERT into `audit_logs`.
- **Exam submit** → INSERT into `exam_results`, UPDATE `applicants`, INSERT into `interview_queue` (if passed), UPDATE `interview_slots`, INSERT into `notifications` + `audit_logs`.
- **Interview evaluate** → UPDATE `interview_queue`, UPDATE `applicants`.
- **Result release** → INSERT into `admission_results`, UPDATE `applicants`, INSERT into `notifications` + `audit_logs`.
- **Cancel & Move** → UPDATE `applicant_exam_slots` (or `interview_queue`), UPDATE `exam_slot_schedule` (or `interview_slots`), INSERT into `notifications` + `audit_logs`.

5.4 What happens during insert / update / delete

- **Insert** → row appears immediately; downstream automation (e.g. `notify_stage_transition`) runs after `COMMIT`.
 - **Update** → guarded by `WHERE` clauses that prevent illegal transitions; UNIQUE constraints catch duplicates.
 - **Delete** → CASCADE for owned data; SET NULL for actor refs; RESTRICT for reference data.
-

6. Status Flow / Life Cycle

6.1 The `overall_status` state machine

From state	Triggered by	New state
<code>pending</code>	Email verified + first login on <code>/student/documents</code>	<code>documents</code>
<code>documents</code>	All required documents uploaded + student clicks Submit	<code>submitted</code>
<code>submitted</code>	Last required document approved by staff	<code>exam</code>
<code>exam</code>	Student passes the entrance exam	<code>interview</code>
<code>interview</code>	Professor evaluates the interview (Pass or Decline)	<code>released</code>
any	Student withdraws OR Admin runs Close Admissions	<code>withdrawn</code>

6.2 Per-status reference

- `pending`: account created, email pending verification.
- `documents`: uploading.
- `submitted`: awaiting SSO review.
- `exam`: docs approved, exam pending or completed but not yet passed.
- `interview`: exam passed, awaiting interview evaluation.
- `released`: interview evaluated; `admission_results` row exists once the Dean clicks Accept or Reject.
- `withdrawn`: terminal.

6.3 Rejected flow

A student can be released as `rejected` (failed exam, declined by Professor + Dean, or override). They land on `released` with `admission_results.result = 'rejected'`. Acceptance suggestions are no longer applicable.

6.4 Cancelled flow

A student can be moved out of an exam or interview slot via **Cancel & Move**. Their `applicant_exam_slots.slot_id` (or `interview_queue.slot_id`) is updated; their `overall_status` does **not** change.

7. Validation

7.1 Registration

- Required: First name, Last name, Birthdate, Sex (M / F), Street + Barangay (must match the hardcoded 30-barangay list), Phone, Email, Password (≥ 8 chars), Applicant type, Course, SHS strand (Freshmen only), hCaptcha.
- Email must be unique.
- Course cap (`accepted_count < max_slots`).
- Strand compatibility for Freshmen.
- Duplicate triplet check (`first_name + last_name + birthdate`).

7.2 Document upload

- PDF, JPG, PNG, WEBP only.
- ≤ 5 MB per file.
- Per-document slot uniqueness (UNIQUE on `(applicant_id, document_type)`).

7.3 Exam

- Access code must match and be within the 5-minute window.
- Per-question type validation (required vs optional, value vs free text).

7.4 Password (anywhere it's set)

- ≥ 8 characters.

7.5 Business rules

- Course cap re-checked at release time.
- Override-reason required when the Dean contradicts the Professor.
- Close Admissions requires a non-empty reason.

7.6 What happens if validation fails

- Render the form with the user's input preserved, an inline error flash, and field-level highlighting.
 - The server never silently accepts invalid input.
-

8. Error Handling

8.1 Database errors

- `PDOException` is caught; transaction is rolled back; user-facing flash is generic (“Something went wrong. Please try again.”).
- Detailed error is logged to `php-error.log`.

8.2 Invalid inputs

- Inline validation error on the field; submission rejected.

8.3 Unauthorized access

- `Auth::requireRole()` redirects with a permission flash.
- The page itself never renders.

8.4 Session expiration

- Student dashboard: next request redirects to `/login` with “Your session has expired.”
- AJAX call: returns 401 + JSON error; client-side script shows a toast.
- Mid-exam expiry: autosave fails (401); student is prompted to log in again; on return, exam draft is restored.

8.5 Missing records

- 404 page rendered; flash explains the missing entity.

8.6 Race conditions

- UNIQUE constraints + `FOR UPDATE` locks make second-writer scenarios fail safely (SQL state `23000` is caught and presented as “This applicant already has a result.”).

8.7 Network issues

- Forms with autosave (exam) buffer locally; resubmission picks up where it left off.

8.8 File upload failures

- Upload rolled back; document row stays at `pending`.
 - Partial files in `tmp/` cleaned up by PHP.
-

9. Security Features

9.1 Password hashing

- bcrypt at cost 12 via `password_hash()`.
- `password_verify()` on login.

9.2 Session security

- Regenerate session ID on login.
- `HttpOnly`, `SameSite=Strict`, `Secure` (in production) on session cookie.
- 30-minute timeout for students; 2-hour timeout for staff.

9.3 Login rate limiting

- 5 failed attempts in 15 minutes → 15-minute logout per email.

9.4 SQL injection prevention

- 100% prepared statements via PDO.

9.5 XSS prevention

- `htmlspecialchars()` (or shorthand `e()` helper) on every user-controlled string in templates.

9.6 CSRF protection

- `csrf_field()` on every form.
- Server-side token verification on every POST.

9.7 Input sanitization

- Names uppercased, email lowercased, phone normalized to digits, address trimmed.

9.8 Audit logs

- Every state change writes to `audit_logs` (actor, action, description, entity, entity_id, IP, timestamp).
- Append-only by application convention.

9.9 Other defenses

- hCaptcha on login + registration.
- Content Security Policy header.
- `X-Frame-Options: DENY`.
- `X-Content-Type-Options: nosniff`.

- HTTPS required in production.

10. User Guide

10.1 Student Guide

1. Go to `/register`. Fill in the form. Submit.
2. Check your email. Click the magic link OR enter the 6-digit code at `/verify-pending`.
3. You're logged in and on `/student/documents`. Upload each required document one at a time.
4. Click **Submit Application** when all docs are uploaded.
5. Wait for the SSO to approve your documents. You'll get an email when they do.
6. Once approved, you'll see your assigned exam slot on `/student/exam`. If you can't make it, click **Request Reschedule**.
7. On exam day, log in, enter the access code (read aloud by the Proctor), and take the exam.
8. If you pass, you'll be auto-assigned to an interview slot. Visit `/student/interview` for the date, time, location, and your live queue position. If you need to reschedule, click the Need to reschedule? button.
9. After the interview, watch your inbox for the result.
10. If accepted, log in and click **Confirm Enrollment** within 7 days.

10.2 SSO Guide

1. Log in. Land on `/admin/dashboard`.
2. Build the exam at `/staff/exam` if not yet done.
3. Add exam rooms at `/staff/exam/slots`.
4. Add interview sessions at `/staff/interviews/setup` (if you also own this — Staff / Dean usually owns it per college).
5. Open `/staff/applicants` to review documents. Approve or Request Resubmission per document.
6. As applicants advance through exam and interview, monitor progress on the dashboard.
7. Handle exam reschedules at `/staff/exam/reschedule`.
8. If a slot needs to be cancelled, open `/staff/exam/cancel-slot` (or `/staff/interviews/cancel-slot`), pick a replacement, supply a reason, and submit.
9. SSO does **not** release results — that's the Dean's job.

10.3 Dean Guide

1. Log in. Land on `/admin/dashboard`.
2. Adjust `max_slots` or tier thresholds for your college's courses at `/admin/courses`.
3. Monitor applicants in your college as they advance.
4. Review interview reschedule requests at `/staff/interviews/absent?tab=requests`.
5. When applicants land in **Recommended: Accept** or **Recommended: Decline** on `/staff/results`, click Accept or Reject per row. If you override the Professor's recommendation, supply a written reason.
6. Use Bulk Accept / Bulk Reject when you have a batch to release.
7. Watch the per-course slot capacity panel at the top of the page — don't accept more than your `max_slots` per course.

10.4 Staff (Professor) Guide

1. Log in. Land on `/staff/dashboard`.
2. Open `/staff/interviews/setup` to add interview sessions for your college.
3. On interview day, open `/staff/interviews/queue`. The queue is first-come, first-served by `queue_number` — **there is no "Call Next" button**. The next student is whoever sits at the top of the list; once you finish the current applicant, the next moves up automatically.
4. Click the applicant's name to see their profile, exam score, and documents in the side drawer.
5. Click the **Evaluation** button on the row, write notes, pick **Pass** or **Decline**. This is a recommendation — the Dean releases the actual result.
6. No-shows are flagged automatically when the slot ends — there is **no manual "No-show" button** — and the system auto-reschedules them to the next available slot.

10.5 Proctor Guide (NEW)

1. Log in. Land on `/staff/dashboard`.
2. Open **Exam Slots** from the sidebar.
3. Pick today's slot for your room.
4. When students are seated and ready, click **Generate Code**. Read the 6-character code aloud.
5. If more time is needed, click **Extend** (same code, fresh countdown).
6. If a new code is needed (e.g. for a late arrival), click **New** (fresh code, invalidates the previous).

10.6 Admin Guide

The Admin can do everything the SSO / Dean / Proctor / Staff can do, plus:

1. `/admin/users` to provision and manage accounts.
 2. `/admin/school-year` to open / close admissions.
 3. `/admin/courses` to add / edit / delete courses.
 4. `/admin/settings` to toggle automations and edit branding.
 5. `/admin/audit-log` to review every action in the system.
 6. **Auto Release** at the top of `/staff/results` to bulk-release every Recommended row.
 7. **Close Admissions** to irreversibly bulk-reject every unreleased applicant with a recorded reason.
-

11. Notes and Special System Behaviour

11.1 Automatic processes

- All-docs-approved → status flips to `exam` + slot assigned + notification.
- New exam slot created → backfill sweep.
- Exam pass → status flips to `interview` + slot assigned + notification.
- New interview session created → bulk assignment sweep.
- Interview evaluation → status flips to `released` (the bucket follows the Pass / Decline outcome).
- No-show: auto-detected on every queue page load; auto-rescheduled if `auto_reschedule_noshows = 1`.
- Accepted no-confirm: auto-withdrawn after 7 days.
- Every state-changing action: audit log row + (for status changes) in-app + email notification.

11.2 Toggles

All under `/admin/settings`:

- `auto_assign_exam_slots` (default 1).
- `auto_reschedule_noshows` (default 1).
- `auto_release_results` (default 0).
- `auto_promote_waitlist` (deprecated no-op).

11.3 Timestamp behaviour

- All timestamps are MySQL `DATETIME` stored in server local time (Asia / Manila).
- Display uses the same timezone.

- No UTC conversion today — a planned hardening step.

11.4 Audit trail

- Every important action: actor, action, description, entity, entity_id, IP, timestamp.
- Append-only by application convention.
- Visible to Admin at `/admin/audit-log`.

11.5 Soft delete behaviour

- Users: `is_active = 0` (soft delete). Linked applicant remains; historical actions are preserved.

11.6 Backup behaviour

- `mysqldump --single-transaction` is the recommended path. No automated scheduling today (gap).

11.7 Scheduling rules

- Exam slots: per-slot opens / closes time, default close = opens + 90 min if blank.
 - Interview sessions: start / end time, capacity (default 30).
 - Cap caps: per-day department cap 3000 (`MAX_EXAM_DAILY_PER_DEPT`).
-

12. Edge Cases and Scenarios

12.1 User submits duplicate data

- Same `first_name + last_name + birthdate` → blocked.
- Same email → blocked by UNIQUE constraint.

12.2 User edits an approved record

- Documents: once approved, the student cannot re-upload that document unless an SSO / Admin Requests Resubmission first.
- Exam answers: immutable after submission.

12.3 Admin deletes an active record

- Delete a user: `is_active = 0` (soft delete). Linked applicant remains; historical actions are preserved.
- Hard-delete a user: Admin can in the database, which CASCADES (and is not recommended).
- Delete a course: Admin gets a warning. If applicants are linked, they remain (FK is on `course_applied` as a string).

12.4 Rejected user reappplies

- Same school year + same email: blocked by the UNIQUE email constraint.
- Workaround: admin manually deletes the old applicant row or migrates it to a new school year.

12.5 Schedule conflicts

- Two interview slots overlap for the same staff: allowed (the system doesn't prevent it). Staff should check their schedule before creating sessions.
- Two students try to book the last available slot at the same time: `FOR UPDATE` locking ensures only one succeeds; the other gets the next-best slot or none.

12.6 Missing requirements

- A student tries to Submit without all required docs uploaded: button is disabled in UI; server-side check rejects.

12.7 Interrupted transactions

- Mid-transaction crash: InnoDB redo log + crash recovery rolls back uncommitted changes.
- Half-uploaded file: browser interruption leaves a partial file in `tmp/`; PHP cleans it up; document row stays at `pending`.

12.8 Expired sessions

- See Section 8.4.

12.9 Simultaneous updates

- Two staff approving same doc: UPDATE is idempotent.
- Two staff releasing same result: UNIQUE constraint on `admission_results.applicant_id` makes the second insert fail.
- Two students racing for the last interview slot: `SELECT ... FOR UPDATE` serializes them.

12.10 Course full while student is mid-process

- Student passes exam but course cap is hit during their interview phase: at result release, staff sees a "Course cap reached" warning. Admin can override (raise the cap) or staff declines the release.

12.11 Student changes their mind about course

- Before submission: change applicant type / course is allowed before SSO has approved any document.

- After submission: only possible if SSO / Dean / Admin suggests an alternative via the course-suggestion flow.

12.12 Staff account department changed

- If admin reassigns a staff member's department (e.g., from CCS to CON), their existing assigned queue rows remain (no auto-reassign). Going forward, new applicants go to the new department's pool.

12.13 Dean overrides the Professor's recommendation

- Dean clicks Accept on a Recommended: Decline row (or vice versa).
- Override-reason modal pops; reason is required.
- Reason stored on `admission_results.remarks` and logged as `admission_result_override`.

12.14 Emergency cancels an exam or interview slot

- SSO / Admin opens `/staff/exam/cancel-slot` (or SSO / Dean / Admin opens `/staff/interviews/cancel-slot`), picks a replacement slot, supplies a reason, submits.
 - Every applicant in the cancelled slot is moved in a single locked transaction; per-student notifications fire.
-

13. Reports and Analytics

13.1 Available reports

Report	Path	Description
Applicant Funnel	<code>/admin/dashboard</code>	Chart.js bar chart: count of applicants per status.
Applicants by Course	<code>/admin/dashboard</code>	Counts grouped by <code>course_applied</code> .
Applicants Export	<code>/admin/dashboard?export=csv</code>	CSV with filters.
Results Export	<code>/admin/results?export=csv</code>	CSV of all admission decisions.
Audit Log	<code>/admin/audit-log</code>	Read-only log view with filters.
Today's Interview Queue	<code>/staff/interviews/queue</code>	Live operational view.
Course Cap Status	<code>/admin/courses</code>	Per-course <code>max_slots</code> vs <code>accepted_count</code> .

13.2 Filters

- Date range (`created_at` or `updated_at`).
- `overall_status` .
- `course_applied` .
- `school_year` .
- `applicant_type` .
- `department` (for staff / dean).

13.3 Export formats

- CSV (UTF-8, comma-separated). Header row included.
- No PDF report generation today (gap; recommended for admission letters).

13.4 Role access

Report	Admin	SSO	Dean (own college)	Staff / Proctor	Student
Applicant Funnel	yes	yes	yes	no	no
Applicants Export	yes	yes	yes	no	no
Results Export	yes	no	yes	no	no
Audit Log	yes	no	no	no	no
Interview Queue	yes	yes	yes (own)	yes (own queue)	no
Course Cap Status	yes	yes	yes	no	no

14. Final Summary

14.1 Overall workflow

Register → Verify email → Login → Upload docs → SSO approves → Auto-assigned exam → Pass exam → Auto-assigned interview → Pass / Decline interview → Dean releases → Confirm enrollment.

Each transition is triggered by a specific event (form submission, staff approval, exam pass) and produces a deterministic set of side effects (status update, audit log, notification, downstream automation).

14.2 Security flow

bcrypt cost 12 + per-email login lockout + hCaptcha on registration / login + email verification + session regeneration on login + CSRF tokens on every form + 100% prepared statements + CSP + HTTPS in production + audit logging.

14.3 User interaction flow

- **Students** see only their own data.
- **Staff (Professors)** see only their own interview queue.
- **Proctors** see only their own college's exam rooms.
- **Deans** see their college's applicants for interview + results.
- **SSO** sees all applicants (system-wide doc / setup queue) but no Results.
- **Admin** sees everything + Auto Release + Close Admissions.

14.4 Approval flow

- **Documents:** SSO approves / requests resubmission → auto-advance applicant to exam when all approved.
- **Exam:** auto-graded by the system → pass / fail decision is automatic.
- **Interview:** Staff evaluates → Pass / Decline recorded (recommendation only).
- **Result:** Manual (single / bulk) or automatic (sweep) by the Dean
 - Admin. Override of the Professor's recommendation requires a written reason. Close Admissions (Admin) is an irreversible bulk-reject of every unreleased row.

14.5 Database flow

- 26 schema-declared InnoDB tables + 5 on-demand auxiliaries.
- ~28 FOREIGN KEY constraints (CASCADE for owned data, SET NULL for actor references).
- ~13 UNIQUE constraints — the integrity backbone (`uq_email` , `uq_applicant_active` , `uq_applicant_result` , `uq_aes_applicant`).
- All multi-step changes wrapped in `BEGIN` / `COMMIT` / `ROLLBACK` .
- `SELECT ... FOR UPDATE` on hot paths (interview slot assignment, exam slot Cancel & Move, interview slot Cancel & Move) with deterministic lock order.

14.6 Important notes for presenters

- **Triggers / stored procedures:** intentionally not used. All business logic is in PHP (`core/automation.php`) for portability and testability. The DB enforces integrity; the app enforces business rules.
- **Concurrency:** safe by design — UNIQUE constraints + `FOR UPDATE`
 - short transactions + try / catch with rollback.
- **Deadlock prevention:** every lock acquired in a deterministic order; transactions are short; UNIQUE indexes act as a safety net.
- **Auditability:** every state change is in `audit_logs` with actor, IP, timestamp.
- **Role redesign:** Dean releases results (not SSO). Interview outcome is Pass / Decline (not Pass / Fail). Override demands a written reason. Proctor is a new, scoped role for exam day.
- **Demo seed:** `database/seed_demo.sql` provides sample applicants spread across all funnel stages, idempotent, dates anchored to `CURDATE()` so today's queue is always populated.

14.7 Known gaps (for transparency)

- Audit logs are append-only by convention, not by DB trigger. (Fix: add an `AFTER UPDATE` trigger blocking changes.)

- No PDF report generation (admission letters).
 - No 2FA for staff / admin.
 - Mobile UX for exam interface is rough.
 - No automatic backup scheduling — operationally must run `mysqldump` on a schedule.
 - OCR-style document auto-validation pipeline was removed in this build; documents now rely entirely on human review.
-

Document version: 2.0 · Verified against `baseline` after the latest project zip merge

- Source of truth is always the code.